

Test Cases

โปรแกรม **Emergency Call Dispatch Queue** ต้องทดสอบการทำงานในหลายสถานการณ์ เพื่อให้มั่นใจว่า Queue สามารถจัดการ Event ได้ถูกต้องทั้งกรณีปกติและกรณีพิเศษ

ตาราง Test Cases

กรณีทดสอบ	สถานการณ์ทดสอบ	ข้อมูลนำเข้า / การทำงาน	ผลลัพธ์ที่คาดหวัง
TC1	กรณีปกติ	เพิ่ม E1, E2, E3 เข้า FIFO Queue	ประมวลผลตามลำดับ E1 → E2 → E3
TC2	Queue ว่าง	เรียก dequeue() ขณะที่ Queue ว่าง	ไม่เกิดข้อผิดพลาด และ Queue ยังคงว่าง
TC3	มีข้อมูล 1 รายการ	เพิ่ม E1 เพียงรายการเดียว แล้วทำ dequeue()	ประมวลผล E1 และ Queue กลายเป็นว่าง
TC4	Queue ขนาดใหญ่	เพิ่ม Event จำนวน 10,000 รายการ	สามารถจัดเก็บและประมวลผล Event ทั้งหมดได้
TC5	Priority เท่ากัน	เพิ่ม E3 และ E5 ซึ่งมีระดับ Critical เท่ากัน	E3 → E5 เพราะ E3 เข้ามาก่อน
TC6	ยกเลิก Event ที่มีอยู่	ยกเลิก E3 จาก [E1,E2,E3,E4,E5,E6]	E3 ถูกลบออกสำเร็จ
TC7	ยกเลิก Event ที่ไม่มีอยู่	ยกเลิก E99	แสดงข้อความ ไม่พบ Event E99 และข้อมูลเดิมไม่เปลี่ยน
TC8	การจัดลำดับตาม Priority	เพิ่ม Event ที่มี Critical, High, Medium และ Low	ประมวลผลตามลำดับ Critical → High → Medium → Low

รายละเอียดระดับความรุนแรง

ระดับความรุนแรง	ค่า Priority	ความหมาย
Critical	1	ร้ายแรงที่สุด
High	2	ร้ายแรง
Medium	3	ปานกลาง
Low	4	ต่ำ

หมายเหตุ: ใน Priority Queue ค่า Priority ยิ่งน้อยยิ่งมีความสำคัญสูง และถ้า Priority เท่ากัน จะใช้ลำดับการเข้าคิว (Sequence) เป็นตัวตัดสินว่าใครมาก่อน

TC1: Normal Case

Purpose

ทดสอบการทำงานปกติของ FIFO Queue

Input

E1
E2
E3

Queue

[E1, E2, E3]

Operation

DEQUEUE
DEQUEUE
DEQUEUE

Expected Result

E1 → E2 → E3

Explanation

FIFO ใช้หลักการ **First In First Out** ดังนั้น Event ที่เข้ามาก่อนต้องถูกประมวลผลก่อน

TC2: Empty Queue

Purpose

ทดสอบกรณีที่ Queue ไม่มีข้อมูล

Initial Queue

[]

Operation

```
queue.poll();
```

Expected Result

null

และ Queue ยังคงเป็น

[]

Explanation

เมื่อ Queue ว่าง poll() จะไม่เกิด Error แต่จะคืนค่า null

กรณีนี้ช่วยตรวจสอบว่าโปรแกรมสามารถจัดการ **Empty Queue** ได้อย่างปลอดภัย

TC3: Single Event

Purpose

ทดสอบกรณีที่ มี Event เพียงรายการเดียว

Input

E1

Queue

[E1]

Operation

DEQUEUE

Expected Result

DEQUEUE → E1

หลังจากนั้น

[]

Explanation

ตรวจสอบว่า Queue สามารถจัดการกรณีที่ มีข้อมูลเพียง 1 รายการได้ถูกต้อง

TC4: Large Queue

Purpose

ทดสอบประสิทธิภาพเมื่อมีข้อมูลจำนวนมาก

Input

10,000 Events

ตัวอย่าง

E1

E2

E3

...

E10000

Operation

ENQUEUE 10,000 Events

DEQUEUE 10,000 Events

Expected Result

All 10,000 Events are processed successfully.

Final Queue = []

Explanation

ใช้ตรวจสอบว่า Queue สามารถรองรับข้อมูลจำนวนมากได้โดยไม่เกิดปัญหาในการจัดเก็บหรือประมวลผล

TC5: Equal Priority

Purpose

ทดสอบกรณีที่ Event มี Priority เท่ากัน

ตัวอย่าง

E3 = Critical = Priority 1

E5 = Critical = Priority 1

แต่

E3 Sequence = 3

E5 Sequence = 5

Queue

E3 → E5

Expected Result

DEQUEUE → E3

DEQUEUE → E5

Explanation

เมื่อ Priority เท่ากัน โปรแกรมจะใช้ sequence เป็นตัวตัดสิน

ดังนั้น Event ที่เข้ามาก่อนจะถูกประมวลผลก่อน

E3 (Sequence 3)



E5 (Sequence 5)

จึงได้

E3 → E5

TC6: Cancel Event

Purpose

ทดสอบการยกเลิก Event ที่มีอยู่ใน Queue

Initial Queue

[E1, E2, E3, E4, E5, E6]

Action

Cancel E3

Expected Result

[E1, E2, E4, E5, E6]

และแสดง

Event cancelled successfully: E3

Explanation

ตรวจสอบว่าโปรแกรมสามารถค้นหาและลบ Event ที่ต้องการออกจาก Queue ได้

TC7: Cancel Non-existing Event

Purpose

ทดสอบการยกเลิก Event ที่ไม่มีอยู่ใน Queue

Initial Queue

[E1, E2, E4, E5, E6]

Action

Cancel E99

แต่ไม่มี E99 ใน Queue

Expected Result

Event not found: E99

Queue ยังคงเป็น

[E1, E2, E4, E5, E6]

Explanation

โปรแกรมต้องไม่ลบข้อมูลอื่น และต้องแจ้งให้ผู้ใช้ทราบว่าไม่พบ Event ที่ต้องการยกเลิก

TC8: Priority Case

Purpose

ทดสอบว่า Priority Queue สามารถจัดลำดับตามความรุนแรงได้ถูกต้อง

Input

E1 = Medium

E2 = Low

E3 = Critical

E4 = High

Priority

Event	Severity	Priority
E1	Medium	3
E2	Low	4
E3	Critical	1
E4	High	2

Expected Result

E3 → E4 → E1 → E2

Explanation

Priority Queue จะเลือก Event ที่มี Priority สูงที่สุดก่อน

Critical → High → Medium → Low

ดังนั้น

E3 → E4 → E1 → E2

สรุป Test Cases

TC1 : Normal Case

TC2 : Empty Queue

TC3 : Single Event

TC4 : Large Queue

TC5 : Equal Priority

TC6 : Cancel Existing Event

TC7 : Cancel Non-existing Event

TC8 : Priority Case

โดย TC5 และ TC7 สำคัญ เพราะเป็นกรณี Edge Case ที่แสดงให้เห็นว่าโปรแกรมไม่ได้ทดสอบแค่การทำงานปกติ แต่รองรับกรณีพิเศษด้วย